

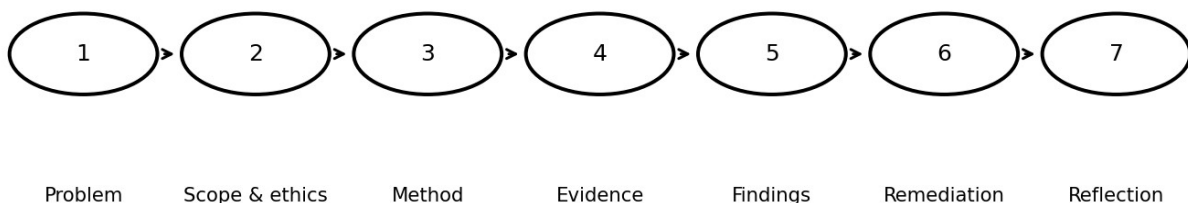
BUILDING A CYBERSECURITY PORTFOLIO ON GITHUB

*A practical guide to repositories, case studies, evidence, security,
accessibility, and career presentation*

Alberto (Al) Leiva

Version 1.0 - July 2026

A strong cybersecurity portfolio tells a complete story



Show what you did, why it mattered, how you stayed authorized, and what you learned.

CC BY-NC-SA 4.0

Purpose and responsible-use notice

This manual helps learners and professionals build a public cybersecurity portfolio without exposing sensitive information or misrepresenting experience. It is intended for lawful, authorized, and ethical work.

Safety rule: *Never publish confidential information, real credentials, exploit details for unpatched systems, or evidence from work that you do not have permission to disclose.*

Table of contents

- 1. Why a cybersecurity portfolio matters
- 2. Safety, authorization, privacy, and ethics
- 3. Define your target role and portfolio strategy
- 4. Create and secure your GitHub account
- 5. Build a professional profile README
- 6. Repository architecture and naming
- 7. Write a README that proves professional judgment
- 8. Project 1 - Cybersecurity risk assessment
- 9. Project 2 - SOC alert triage and incident case
- 10. Project 3 - Detection engineering
- 11. Project 4 - Cloud security review
- 12. Project 5 - AI security and governance
- 13. Project 6 - Policy, standard, and procedure
- 14. Evidence without exposing sensitive information
- 15. Git, branches, issues, and pull requests
- 16. GitHub Actions and secure automation
- 17. Repository security features
- 18. Accessibility and inclusive documentation
- 19. Publish with GitHub Pages and releases
- 20. Quality review before publication
- 21. Resume, LinkedIn, and interview integration
- 22. 30-, 60-, and 90-day plan
- 23. Templates and checklists
- 24. Official references

Recommended portfolio repository structure

cybersecurity-portfolio/

```
|— README.md
|— 01-risk-assessment/
|— 02-detection-engineering/
|— 03-incident-response/
|— 04-cloud-security/
|— 05-ai-security/
|— docs/
|— LICENSE
└— SECURITY.md
```

1. Why a cybersecurity portfolio matters

A resume tells employers what you claim to know. A portfolio shows how you think, document, communicate, and improve. It is especially useful for students, career changers, experienced professionals moving into a new specialty, and candidates whose strongest work cannot be described fully on a one-page resume.

A portfolio should not imitate a trophy shelf. It should demonstrate judgment. Strong projects explain the business problem, authorization, scope, method, evidence, findings, limitations, recommendations, and lessons learned.

2. Safety, authorization, privacy, and ethics

Only publish work created in an environment you own, an approved laboratory, an authorized training platform, or a professional engagement that explicitly permits publication. Technical ability does not create permission.

Never upload client files, employer data, screenshots containing names, IP addresses, access tokens, private keys, internal architecture, vulnerability details, customer records, or anything covered by a confidentiality agreement.

3. Define your target role and portfolio strategy

Choose a target role before building projects. A SOC analyst portfolio should look different from a GRC, cloud security, penetration testing, digital forensics, AI security, or security architecture portfolio.

Use a simple strategy: three strong projects that match the target role, one cross-functional project that shows business communication, and one learning project that shows curiosity and growth.

Target role	Good portfolio evidence
SOC analyst	Triage cases, timelines, detections, escalation notes
GRC analyst	Risk registers, control tests, policies, evidence maps
Cloud security	Architecture, IAM review, logging, posture findings
AI security	Threat models, RAG tests, agent controls, governance
Security architect	Reference architectures, tradeoffs, control mappings

4. Create and secure your GitHub account

Use a professional username, a recognizable display name, a concise biography, and links to LinkedIn or a personal site. Enable multifactor authentication and protect recovery methods.

Use separate credentials for laboratories and demonstrations. Do not reuse production credentials. Treat public repositories as permanently public even if you later delete content.

5. Build a professional profile README

A profile README appears when a public repository has the same name as your GitHub username. Use it to explain your role, specialties, certifications, selected projects, current learning, languages, and contact method.

Keep the profile readable. Recruiters should understand your value within the first screen. Link to three to six strongest projects instead of listing every exercise.

6. Repository architecture and naming

Use predictable names, numbered folders, and consistent file formats. A reader should know where to begin without opening every file.

Each repository should include a README, license, project files, sanitized evidence, and a clear note about authorized use. For security tools or scripts, add a SECURITY.md file explaining how to report vulnerabilities responsibly.

7. Write a README that proves professional judgment

A project README is often the first and only file a reviewer reads. Write it as a short case study rather than a list of commands.

Include the problem, objective, scope, authorization, environment, architecture, tools, method, evidence, findings, recommendations, limitations, skills demonstrated, and instructions for safe reproduction.

- Project title and one-sentence outcome
- Business or learning problem
- Authorization and scope
- Environment and assumptions
- Architecture or workflow diagram
- Tools and data sources
- Method and validation
- Findings and severity
- Recommendations and priorities
- Limitations
- Skills demonstrated
- Safe reproduction steps

8. Project 1 - Cybersecurity risk assessment

Create a fictional small business or nonprofit and perform a structured risk assessment. Document assets, threats, vulnerabilities, likelihood, impact, existing controls, risk treatment, owners, and target dates.

Publish a sanitized risk register, executive summary, heat map, control recommendations, and a short reflection explaining how business priorities changed your recommendations.

9. Project 2 - SOC alert triage and incident case

Use an authorized lab or sample logs to investigate a phishing, suspicious sign-in, malware, or impossible-travel alert. Build a timeline and explain the evidence that supports or weakens each hypothesis.

Publish a case summary, timeline, indicators, MITRE ATT&CK mapping, containment recommendations, escalation decision, and lessons learned. Do not publish real victim data.

10. Project 3 - Detection engineering

Create a detection rule from a defined threat behavior. Explain the data source, fields, logic, expected false positives, tuning decisions, validation method, and response guidance.

Examples include Sigma rules, Microsoft Sentinel analytics, Splunk searches, Elastic rules, or Wazuh detections. Include test data that is synthetic or approved.

11. Project 4 - Cloud security review

Build a small Azure, AWS, or Google Cloud laboratory and review identity, logging, network exposure, storage permissions, encryption, secrets, and monitoring.

Publish an architecture diagram, findings table, evidence screenshots with sensitive details removed, remediation plan, and mapping to a recognized benchmark or cloud security framework.

12. Project 5 - AI security and governance

Design a fictional AI assistant, RAG system, or agent. Document its data flows, threat model, prompt-injection risks, tool permissions, privacy controls, human oversight, monitoring, and incident response.

Publish an AI system inventory record, risk assessment, security architecture, test plan, evaluation results, and governance checklist.

13. Project 6 - Policy, standard, and procedure

Write a practical policy package for a fictional organization. Good options include acceptable use, access control, incident response, third-party risk, AI use, vulnerability management, or data classification.

Show the relationship between policy, standard, procedure, evidence, and ownership. Avoid copying templates without adapting them to a business context.

14. Evidence without exposing sensitive information

Evidence can include diagrams, sanitized screenshots, sample logs, synthetic datasets, command output, configuration excerpts, test results, and before-and-after comparisons.

Remove usernames, tenant names, IP addresses, tokens, keys, email addresses, device IDs, customer names, geolocation, and proprietary details. When uncertain, recreate the evidence with synthetic data.

Publish	Do not publish
Synthetic logs	Real customer logs

Redacted screenshots	Tokens, keys, cookies
Fictional architecture	Internal network diagrams
Approved lab findings	Uncoordinated vulnerability details

15. Git, branches, issues, and pull requests

Use commits to show progress and discipline. Write clear messages such as “Add risk treatment table” rather than “update.” Use branches for meaningful changes and pull requests to explain decisions, testing, and review.

Issues can track planned work, defects, research questions, and improvement ideas. A well-maintained project history demonstrates collaboration skills even when you work alone.

16. GitHub Actions and secure automation

Automation can validate Markdown links, run tests, lint code, scan dependencies, build documentation, or publish a GitHub Pages site. Keep workflows simple and explain what they do.

Use least-privilege workflow permissions, pin trusted actions to immutable references when appropriate, protect environments, avoid untrusted input in scripts, and never print secrets to logs.

17. Repository security features

Enable available security features that fit the project, such as dependency alerts, dependency updates, code scanning, secret scanning, and security policies. Availability can vary by repository type and plan.

Treat any exposed credential as compromised. Revoke or rotate it immediately; deleting it from the latest commit is not enough because Git history may retain it.

18. Accessibility and inclusive documentation

Use meaningful headings, descriptive links, alt text for diagrams, readable contrast, plain language, captions, and text alternatives for visual evidence. Avoid relying on color alone to communicate risk.

Provide DOCX or PDF versions when useful, but keep Markdown as the accessible source of truth. Explain abbreviations and offer keyboard-friendly navigation.

19. Publish with GitHub Pages and releases

GitHub Pages can turn a repository into a static portfolio site. Use it for a clean landing page, project summaries, and navigation to repositories.

Use releases for stable versions of manuals, templates, datasets, or tools. Attach only sanitized artifacts and use version numbers consistently.

20. Quality review before publication

Before publishing, verify every link, filename, screenshot, instruction, and claim. Ask whether a reader can reproduce the work safely, understand limitations, and distinguish facts from assumptions.

Run spelling and grammar checks, validate code, test downloads, inspect mobile readability, and ask a trusted reviewer to check both technical accuracy and clarity.

21. Resume, LinkedIn, and interview integration

Add a “Selected Projects” section to the resume with a one-line outcome and direct repository link. On LinkedIn, feature the strongest repositories and explain the problem solved rather than merely announcing an upload.

In interviews, use the portfolio to describe decisions, tradeoffs, failures, and improvements. Never claim a lab exercise was production experience.

22. 30-, 60-, and 90-day plan

In the first 30 days, secure the account, create the profile README, choose a target role, and publish one polished project. By day 60, add two role-aligned projects and improve documentation. By day 90, publish a cross-functional project, request feedback, and integrate the portfolio into applications and interviews.

Quality matters more than volume. Maintain and improve existing work rather than creating dozens of unfinished repositories.

Period	Deliverable
Days 1-30	Profile README and one polished project
Days 31-60	Two role-aligned projects and review improvements
Days 61-90	Cross-functional project, feedback, job integration

23. Templates and checklists

Project README template

- Title and outcome

- Problem
- Authorization and scope
- Environment
- Architecture
- Tools
- Method
- Evidence
- Findings
- Recommendations
- Limitations
- Lessons learned
- References
- License

Pre-publication checklist

- ☐ No secrets or credentials
- ☐ No confidential or personal data
- ☐ Authorization documented
- ☐ Links tested
- ☐ Files open correctly
- ☐ Spelling and grammar reviewed
- ☐ Alt text included
- ☐ Claims supported by evidence
- ☐ Limitations stated
- ☐ License selected
- ☐ Contact method appropriate

24. Official references

[GitHub profile README](#)

[Creating and managing repositories](#)

[GitHub Pages](#)

[Quickstart for securing a repository](#)

[Secret scanning](#)

[Code scanning alerts](#)

[Security for GitHub Actions](#)

[Artifact attestations](#)

License and attribution

Copyright © 2026 Alberto (Al) Leiva. Licensed under Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International.